Exercise-1: Programs on Input/output, Class & Object, Type Conversion and Wrapper

1.a) Classes: Write a Java program to create Class as Registration with properties as Full Name(String) ,Gender(char), Age(int), Height(double), Phone Number(long), and isMarried(Boolean) and print their values.

Aim: Create a Java class named **Registration** that models a real-world entity (a person registering) with properties such as full name, gender, age, height, phone number, and marital status, and to display their values.

Description: Registration class models a person's registration details with properties such as full name, gender, age, height, phone number, and marital status. It includes a constructor to initialize these properties and a method to display them. This class is useful for capturing and showing personal information during a registration process. Properties: Stores personal details. Shows the details on the console.

Program:

```
import java.util.Scanner;
class Registration {
        // Properties
        String fullName;
        char gender;
        int age;
        double height;
        long phoneNumber;
        boolean isMarried;

        // Method to print the values of the properties
        public void printDetails() {
            System.out.println("Full Name: " + fullName);
            System.out.println("Gender: " + (gender == 'M' ? "Male" : "Female"));
            System.out.println("Age: " + age);
            System.out.println("Height: " + height);
            System.out.println("Phone Number: " + phoneNumber);
            System.out.println("Marital Status: " + (isMarried ? "Married" : "Not Married"));
        }
    }

    public class Main {
        public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);

        // Create an instance of Registration
        Registration registration = new Registration();

        // Prompt user for input and assign values to the instance
        System.out.print("Enter Full Name: ");
        registration.fullName = scanner.nextLine();

        System.out.print("Enter Gender (M/F): ");
        registration.gender = scanner.next().charAt(0);

        System.out.print("Enter Age: ");
        registration.age = scanner.nextInt();

        System.out.print("Enter Height: ");
        registration.height = scanner.nextDouble();

        System.out.print("Enter Phone Number: ");
        registration.phoneNumber = scanner.nextLong();

        System.out.print("Enter Marital Status (true for married, false for not married): ");
        registration.isMarried = scanner.nextBoolean();

        // Print the details
        registration.printDetails();

        scanner.close();
    }
}
```

Input :

```
Full Name: Tadiparti  Hareesh
Gender: M (Male)
Age: 22
Height: 5.9
```

Phone Number: 9876543210

Marital Status: false (not married)

Expected output:

Full Name: Tadiparti Hareesh

Gender: M

Age: 22

Height: 5.9

Phone Number: 9876543210

Marital Status: false

1. b) Write a Java program to implement Type Casting and Conversion.

Aim: Demonstrate type casting (both implicit and explicit) and type conversion in Java using different data types.

Description: Type Casting in Java refers to converting one data type into another. Java supports two types of casting:

1. **Implicit Casting (Widening Conversion)**: Automatic conversion from a smaller type to a larger type.
2. **Explicit Casting (Narrowing Conversion)**: Manually converting from a larger type to a smaller type.

   Type Conversion is different from casting and usually refers to converting a value between different types, like from a string to a number, or between primitive types.

   In the program, we will demonstrate both:

   - Implicit casting (e.g., from int to double).
   - Explicit casting (e.g., from double to int).
   - Type conversion (e.g., converting a string to an integer).

Program:

```
import java.util.Scanner;
public class TypeCastingAndConversion {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Widening (Implicit) Casting Example
        System.out.println("Widening (Implicit) Casting Example:");
        int intNumber = 100;
        double doubleNumber = intNumber;  // Automatically converted to double
        System.out.println("Original int value: " + intNumber);
        System.out.println("After widening, double value: " + doubleNumber);

        // Narrowing (Explicit) Casting Example
        System.out.println("\nNarrowing (Explicit) Casting Example:");
        double largeDouble = 123.45;
        int intFromDouble = (int) largeDouble;  // Explicit conversion (losing decimal part)
        System.out.println("Original double value: " + largeDouble);
        System.out.println("After narrowing, int value: " + intFromDouble);

        // Type Conversion Example: String to int
        System.out.println("\nType Conversion Example: String to int");
        System.out.print("Enter a number (as a string): ");
        String strNumber = scanner.nextLine();
        int convertedNumber = Integer.parseInt(strNumber);  // Convert String to int
        System.out.println("Converted String to int: " + convertedNumber);

        // Type Conversion Example: int to String
        System.out.println("\nType Conversion Example: int to String");
        int intValue = 42;
        String strFromInt = Integer.toString(intValue);  // Convert int to String
        System.out.println("Converted int to String: " + strFromInt);

        // Type Conversion Example: double to String
        System.out.println("\nType Conversion Example: double to String");
        double doubleValue = 3.14159;
        String strFromDouble = Double.toString(doubleValue);  // Convert double to String
```

```
        System.out.println("Converted double to String: " + strFromDouble);

        scanner.close();
    }
}
```

Ouput:

Widening (Implicit) Casting Example:
Original int value: 100
After widening, double value: 100.0

Narrowing (Explicit) Casting Example:
Original double value: 123.45
After narrowing, int value: 123

Type Conversion Example: String to int
Enter a number (as a string): 250
Converted String to int: 250

Type Conversion Example: int to String
Converted int to String: 42

Type Conversion Example: double to String
Converted double to String: 3.14159

1. c) Write a Java program to implement Wrapper Classes.

Aim: Create different wrapper classes

Description: Wrapper classes are used to wrap (encapsulate) primitive data types (such as int, char, double, etc.) into their corresponding **object** types. This is useful when objects are required, for example in collections like ArrayList, which cannot hold primitive types. Java provides a wrapper class for each of the primitive data types:

- int → Integer
- char → Character
- double → Double
- boolean → Boolean
- float → Float
- long → Long
- byte → Byte
- short → Short

Autoboxing:

Autoboxing is the automatic conversion of a **primitive type** to its corresponding **wrapper class object**. This happens implicitly when a primitive value is assigned to a wrapper class reference.

Unboxing:

Unboxing is the automatic conversion of a **wrapper class object** to its corresponding **primitive type**. This happens implicitly when an object of a wrapper class is used in a context that requires a primitive type.

Program:

```
public class WrapperClassExample {
   public static void main(String[] args) {
      // Using wrapper classes for primitive data types

      // Integer wrapper class
      int num = 10;
      Integer numWrapper = Integer.valueOf(num);  // Converting int to Integer
      System.out.println("Integer Wrapper: " + numWrapper);
```

```
        // Double wrapper class
        double d = 3.14;
        Double doubleWrapper = Double.valueOf(d);  // Converting double to Double
        System.out.println("Double Wrapper: " + doubleWrapper);

        // Boolean wrapper class
        boolean flag = true;
        Boolean booleanWrapper = Boolean.valueOf(flag);  // Converting boolean to Boolean
        System.out.println("Boolean Wrapper: " + booleanWrapper);

        // Convert Wrapper class to primitive type
        Integer integerVal = Integer.valueOf(20);
        int primitiveInt = integerVal.intValue();  // Converting Integer to int
        System.out.println("Primitive int: " + primitiveInt);

        // Convert String to Wrapper class
        String str = "100";
        Integer parsedInteger = Integer.parseInt(str);  // Converting String to Integer
        System.out.println("Parsed Integer from String: " + parsedInteger);
    }
}
```

Output:

```
Integer Wrapper: 10
Double Wrapper: 3.14
Boolean Wrapper: true
Primitive int: 20
Parsed Integer from String: 100
```

Viva Questions:

1. What are wrapper classes in Java?
2. Why do we use wrapper classes?
3. What is the difference between primitive data types and wrapper classes?
4. What is auto-boxing and how does it work?
5. What is un-boxing and how does it work?
6. Can you give an example of auto-boxing in Java?
7. Can you give an example of un-boxing in Java?
8. What are some commonly used wrapper classes in Java?
9. How do you convert a wrapper class object back to its primitive type?
10. What is the benefit of using wrapper classes in Java collections?
11. How can you convert a String to a Integer using a wrapper class?
12. What will happen if you try to unbox a null value in a wrapper class?
13. How is Integer.parseInt() different from Integer.valueOf()?
14. Can wrapper classes store null values? If yes, explain.
15. What does the intValue() method do in the Integer wrapper class?

Exercise-2: Programs on Control Statements, Command Line Arguments, Arrays Sample Programs

2. a) Write a Java program to take input as Regd.No and print the branch depending upon the department code in that Regd.No using else-if and switch statements. (Eg Regd. No: 23KD1A0505, 8th character is department Code, 5-CSE, 4-ECE, 3- MECH, 2-EEE etc.

Aim: Write a Java program to take input as Regd.No and print the branch depending upon the department code in that Regd.No using else-if and switch statements. (EgRegNo:19KD1A0505, 8th character is department Code, 5-CSE, 4-ECE, 3-MECH, 2-EEE etc.

Description: Program takes a Regd.No (Registration Number) as input, which follows a specific format. The program uses the 8th character in the registration number to determine the department code. Based on this code, the program will print the corresponding branch name.

The registration number format is:

- Example: 19KD1A0505

  The 8th character (in this case A) is used to determine the department

1. Using else-if statements: A series of if-else conditions will check the department code and display the branch.

2. Using switch statement: A more efficient way of handling multiple conditions based on the department code.

The department codes and corresponding branches:

5 → CSE (Computer Science Engineering)

4 → ECE (Electronics and Communication Engineering)

3 → MECH (Mechanical Engineering)

2 → EEE (Electrical and Electronics Engineering)

Program:

```
import java.util.Scanner;
public class DepartmentBranch {
   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);

      // Take input for Regd.No
      System.out.print("Enter Regd. No: ");
      String regdNo = scanner.nextLine();

      // Extract the 8th character (department code)
      char deptCode = regdNo.charAt(7);
```

```
        // Using else-if statement
        System.out.println("\nUsing else-if statement:");
        if (deptCode == '5') {
            System.out.println("Branch: Computer Science Engineering (CSE)");
        } else if (deptCode == '4') {
            System.out.println("Branch: Electronics and Communication Engineering (ECE)");
        } else if (deptCode == '3') {
            System.out.println("Branch: Mechanical Engineering (MECH)");
        } else if (deptCode == '2') {
            System.out.println("Branch: Electrical and Electronics Engineering (EEE)");
        } else {
            System.out.println("Branch: Unknown Department Code");
        }

        // Using switch statement
        System.out.println("\nUsing switch statement:");
        switch (deptCode) {
            case '5':
                System.out.println("Branch: Computer Science Engineering (CSE)");
                break;
            case '4':
                System.out.println("Branch: Electronics and Communication Engineering (ECE)");
                break;
            case '3':
                System.out.println("Branch: Mechanical Engineering (MECH)");
                break;
            case '2':
                System.out.println("Branch: Electrical and Electronics Engineering (EEE)");
                break;
            default:
                System.out.println("Branch: Unknown Department Code");
        } scanner.close();
    }
}
```

Input: **Enter Regd. No: 23KD1A0505**

Sample Input:

Regd.No: 19KD1A0505

Sample Output (Using else-if or switch):

Branch: CSE

2. b) Write a Java program to read input integers from Command Line Arguments and print first and second largest numbers.

Aim: Write a Java program to read input integers from Command Line Arguments and print first and second largest numbers

Description:

Program reads integer values from the command line arguments and finds the first and second largest numbers among them.

1. Command Line Arguments: command line arguments are passed as String values. To use them as integers, we need to convert them from String to int.
2. Finding Largest and Second Largest: The program will compare the integers and track the largest and second-largest values during a single pass through the list.The program takes command-line arguments as input.It will then loop through the arguments, convert them to integers, and compare them to find the largest and second-largest numbers.

Program steps:

1. Create a class to accept command-line arguments.
2. Convert the command-line arguments to integers.
3. Sort the numbers or compare them to find the first and second largest.
4. Print the first and second largest numbers.

Program:

```
public class LargestNumbers {
   public static void main(String[] args) {
      // Check if the user has entered any command line arguments
      if (args.length < 2) {
         System.out.println("Please provide at least two integers as command line arguments.");
         return;
      }

      // Convert command line arguments to integers
      int[] numbers = new int[args.length];
      for (int i = 0; i < args.length; i++) {
         try {
            numbers[i] = Integer.parseInt(args[i]);  // Convert each argument to integer
         } catch (NumberFormatException e) {
```

```
            System.out.println("Invalid input. Please enter only integers.");
            return;
        }
    }
    // Initialize variables to find the first and second largest numbers
    int firstLargest = Integer.MIN_VALUE;
    int secondLargest = Integer.MIN_VALUE;

    // Loop through the array to find the largest and second largest numbers
    for (int num : numbers) {
        if (num > firstLargest) {
            secondLargest = firstLargest;
            firstLargest = num;
        } else if (num > secondLargest && num != firstLargest) {
            secondLargest = num;
        }
    }
    // Check if we found a second largest number
    if (secondLargest == Integer.MIN_VALUE) {
        System.out.println("There is no second largest number.");
    } else {
        System.out.println("First largest number: " + firstLargest);
        System.out.println("Second largest number: " + secondLargest);
    }
  }
}
```

Sample Input (Command-line Arguments):

```
java LargestNumbers 12 45 78 23 67
```

Sample Output:

```
First Largest: 78
Second Largest: 67
```

2.c) Write a Java program to take input as Integer array and print even indexed even numbers and odd indexed odd numbers

Aim: Write a Java program to take input as Integer array and print even indexed even numbers and odd indexed odd numbers.

Description: Program takes an integer array as input and then prints:

- Even-indexed even numbers (i.e., numbers at indices 0, 2, 4, etc. that are even).
- Odd-indexed odd numbers (i.e., numbers at indices 1, 3, 5, etc. that are odd).

The program loops through the array and checks each element's index:

- For even indices (0, 2, 4, ...), it checks if the number is even.

For odd indices (1, 3, 5, ...), it checks if the number is odd. • Take an integer array as input.Loop through the array. Check the index type (even or odd) and the number's condition (even or odd). Print the numbers that satisfy the conditions

Program:

```
import java.util.Scanner;
public class EvenOddIndexedNumbers {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Take the size of the array as input
        System.out.print("Enter the size of the array: ");
        int size = scanner.nextInt();

        // Declare and take input for the array
        int[] numbers = new int[size];
        System.out.println("Enter " + size + " integers for the array:");
        for (int i = 0; i < size; i++) {
            numbers[i] = scanner.nextInt();
        }

        // Printing even indexed even numbers and odd indexed odd numbers
        System.out.println("Even indexed even numbers:");
        for (int i = 0; i < size; i++) {
            if (i % 2 == 0 && numbers[i] % 2 == 0) {
                System.out.print(numbers[i] + " ");
            }
        }
```

```
        System.out.println("\nOdd indexed odd numbers:");
        for (int i = 0; i < size; i++) {
            if (i % 2 != 0 && numbers[i] % 2 != 0) {
                System.out.print(numbers[i] + " ");
            }
        }

        scanner.close();
    }
}
```

Input:

Enter the size of the array: 6

Enter 6 integers for the array:

10 21 22 33 44 55

Expected Output:

Even indexed even numbers:

10 22 44

Odd indexed odd numbers:

21 33 55

Sample Viva-Voce Questions:

1. What are wrapper classes in Java?
2. Why do we use wrapper classes?
3. What is the difference between primitive data types and wrapper classes?
4. What is auto-boxing and how does it work?
5. What is un-boxing and how does it work?
6. Can you give an example of auto-boxing in Java?
7. Can you give an example of un-boxing in Java?
8. What are some commonly used wrapper classes in Java?
9. How do you convert a wrapper class object back to its primitive type?
10. What is the benefit of using wrapper classes in Java collections?
11. How can you convert a String to a Integer using a wrapper class?
12. What will happen if you try to unbox a null value in a wrapper class?
13. How is Integer.parseInt() different from Integer.valueOf()?
14. Can wrapper classes store null values? If yes, explain.
15. What does the intValue() method do in the Integer wrapper class?

Exercise-3: Programs on Control Statements, Operators, Arrays Sample Programs

3.a) Java Program to Convert Decimal Number to Roman Number

Aim: To convert a given decimal number to its corresponding Roman numeral.

Description: Write a Java program converts a decimal number (integer) into its equivalent Roman numeral.

- Roman numerals are represented using combinations of letters such as I, V, X, L, C, D, and M.
- The program breaks the decimal number into parts and maps each part to its Roman numeral equivalent

Program:

```
import java.util.Scanner;
public class DecimalToRoman {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Take input for decimal number
        System.out.print("Enter a decimal number (1 to 3999): ");
        int decimal = scanner.nextInt();

        // Check if the input is within the valid range (1 to 3999)
        if (decimal < 1 || decimal > 3999) {
            System.out.println("Please enter a number between 1 and 3999.");
        } else {
            // Convert decimal to Roman numeral
            String roman = convertToRoman(decimal);
            System.out.println("Roman Numeral: " + roman);
        }

        scanner.close();
    }

    // Method to convert decimal number to Roman numeral
    public static String convertToRoman(int num) {
        // Roman numerals for the values from largest to smallest
        String[] romanNumerals = {
            "M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"
        };
        int[] decimalValues = {
```

```
        1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1
    };

    StringBuilder roman = new StringBuilder();

    // Iterate through decimal values and build the Roman numeral
    for (int i = 0; i < decimalValues.length; i++) {
        while (num >= decimalValues[i]) {
            roman.append(romanNumerals[i]);
            num -= decimalValues[i];
        }
    }
    return roman.toString();
  }
}
```

Input:

Enter a decimal number: 58

Expected output:

Roman numeral: LVIII

3.b). Write a Java program to check whether given number is Extension number. The extension number is the number which is present in the last digit(s) of its square. (Eg.N=25, 625 is Extension number since it contains 25).

Aim: Write a Java program to check whether given number is Extension number. The extension number is the number which is present in the last digit(s) of its square. (Eg.N=25, 625 is Extension number since it contains 25).

Description: Extension number is a number where the number itself appears at the end (last digit(s)) of its square.

For example, if N = 25, the square of 25 is 625. Since the last two digits of 625 are 25, it is considered an Extension number

Program:

```
import java.util.Scanner;
public class ExtensionNumber {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    // Take input for the number
```

```
        System.out.print("Enter a number: ");
        int number = scanner.nextInt();

        // Call the method to check if the number is an extension number
        if (isExtensionNumber(number)) {
            System.out.println(number + " is an Extension Number.");
        } else {
            System.out.println(number + " is not an Extension Number.");
        }

        scanner.close();
    }

    // Method to check if the number is an extension number
    public static boolean isExtensionNumber(int num) {
        // Find the square of the number
        int square = num * num;

        // Convert both the number and its square to strings
        String numStr = String.valueOf(num);
        String squareStr = String.valueOf(square);

        // Check if the number appears at the end of the square
        return squareStr.endsWith(numStr);
    }
}
```

Input:

Enter a number: 25

Expected Ouput:

25 is an Extension Number.

3.c) Write a Java program to take input as Amount in rupees and print their denominations and total number notes

Aim: Write a Java program to take input as Amount in rupees and print their denominations and total number notes

Description: program takes an amount in rupees as input and then calculates and displays the denominations of currency notes needed to make up that amount. It also prints the total number of notes required

Program:

```
import java.util.Scanner;
public class Denominations {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Take input for the amount
        System.out.print("Enter the amount in rupees: ");
        int amount = scanner.nextInt();

        // Denominations of currency notes
        int[] denominations = {500, 200, 100, 50, 20, 10, 5, 2, 1};

        // Array to store the count of each denomination
        int[] noteCount = new int[denominations.length];

        // Calculate the number of notes for each denomination
        for (int i = 0; i < denominations.length; i++) {
            if (amount >= denominations[i]) {
                noteCount[i] = amount / denominations[i]; // Number of notes of this denomination
                amount = amount % denominations[i]; // Remaining amount
            }
        }

        // Print the denominations and the count of notes
        System.out.println("Denominations and their counts:");
        for (int i = 0; i < denominations.length; i++) {
            if (noteCount[i] > 0) {
                System.out.println(denominations[i] + " rupee notes: " + noteCount[i]);
            }
```

```
        }
        scanner.close();
    }
}
```

Input:

Enter the amount in rupees: 1237

Expected Output:

Denominations and their counts:

500 rupee notes: 2

200 rupee notes: 1

100 rupee notes: 2

20 rupee notes: 1

10 rupee notes: 1

5 rupee notes: 1

2 rupee notes: 1

Sample Viva-Voce Questions:

1. What is a Roman numeral?
2. Explain the logic behind converting a decimal number to Roman numerals.
3. What are the basic symbols used in Roman numerals, and what do they represent?
4. How do you handle numbers greater than 1000 in Roman numeral conversion?
5. What is the purpose of the StringBuilder class in this program?
6. How do you ensure that the largest Roman numeral values are used first in the conversion?
7. Can you modify the program to handle numbers beyond 3999?
8. What would happen if you input a number that is not a valid Roman numeral range (like 4000 or above)?
9. What is an extension number?
10. Explain the logic behind checking if a number is an extension number.
11. How do you extract the last digits of a number in Java?
12. Why do you convert both the original number and its square to strings in this program?
13. What would happen if the square of the number is smaller than the number itself (like 1)?
14. Can you think of any other methods to check if a number is an extension number?
15. Is there any optimization possible in checking extension numbers for larger numbers?
16. What is the purpose of breaking an amount into denominations?
17. Why are we using the int[] denominations array in this program?
18. Explain the logic behind calculating the number of notes for each denomination.
19. What is the role of the totalNotes variable?

20. What would happen if the amount inputted is zero?

21. Can you handle edge cases such as amounts that cannot be exactly divided by any denomination?

22. What changes would you make to allow the program to handle denominations in different currencies?

23. Why do we need to check the condition if (amount >= denom) before calculating the number of notes?

Exercise-4: Programs on Class & Object, Method, Constructor Sample Programs

4.a) Create a Class named Student with properties as Student Id, Student Name, gender, department, Age, Aggregate and methods as insertStudent() for inserting student details and displayStudent() for printing student details.

Aim: The aim of this Java program is to create a class named Student with various attributes and methods to store and manage student details.

Description: This Java program is designed to define a Student class that represents the details of a student. The class includes various properties (attributes) and methods (functions) to store and display student information efficiently.

Program:

```
import java.util.Scanner; class Student
{
String id; String name; String gender;
String department; int age;
Scanner sc=new Scanner(System.in); public void insertStudentDetails()
{
System.out.println("Enter student id"); id=sc.next();
System.out.println("Enter student name"); name=sc.next();
System.out.println("Enter student gender"); gender=sc.next();
System.out.println("Enter student department"); department=sc.next();
System.out.println("Enter student age");      age=sc.nextInt();
}
Public void display()
{
System.out.println(id);
System.out.println(name);
System.out.println(age);
System.out.println(department);
System.out.println(gender);
}
```

```
Public static void main(String args[])
{
Student s=new Student(); s.insertStudentDetails(); s.display();
}
}
```

Input:

Enter Student id: 19KDA1A506

Enter Student Name: MAHI

Enter Student Gender: male

Enter Student Department: CSE

Enter Student Age: 32

Output:

Enter student id 19KDA1A506

Enter student name

MAHI

Enter student gender

male

Enter student departmen CSE

Enter student age 32

19KDA1A506 MAHI

32

CSE

Male

4. b) Create a class Student with same properties as above and create a constructor to insert student details and return the data using toString() method.

Aim: The aim of this program is to create a Student class with attributes such as name, id, age, and grade.

Description: Here is a Java program that defines a Student class with properties such as name, id, age, and grade. It includes a constructor to initialize these properties and overrides the toString() method to return student details.

Program:

```
import java.util.Scanner;
class Student
{
String id;
String name; String gender; String department; int age;
Student(String id, String name, String gender, String department, int age)
{
this.id=id; this.name=name; this.gender=gender;
this.department=department; this.age=age;
}
Public String toString()
{
return id+','+name+','+age+','+department+','+gender;
}
public static void main(String args[])
{
String id; String name; String gender;
String department;
 int age;
Scanner sc=new Scanner(System.in);
System.out.println("Enter student id"); id=sc.next();
System.out.println("Enter student name"); name=sc.next();
System.out.println("Enter student gender"); gender=sc.next();
System.out.println("Enter student department"); department=sc.next();
System.out.println("Enter Student age");
 age=sc.nextInt();
Student s=new Student(id,name,gender,department,age); System.out.println(s);
}
}
```

Input:

Enter Student Name: lasya

Enter Student ID: 123

Enter Student Age: 20

Enter student gender: female

Enter Student department: cse

Output:

Enter student id 123

Enter student name lasya

Enter student gender female

Enter student department

cse

Enter Student age 20

123

Lasya

20

Cse

female

Sample Viva-Voce Questions:

1: What is the purpose of the Student class in this program?

The purpose of the Student class is to represent a **student entity** with properties like studentId, studentName, gender, department, age, and aggregate. It provides methods like insertStudent() for inserting details and displayStudent() for printing the stored information.

2: What are the attributes (properties) defined in the Student class?

- int studentId → Unique identifier for a student.
- String studentName → Stores the name of the student.
- String gender → Stores the gender (Male/Female/Other).
- String department → Stores the department name.
- int age → Stores the age of the student.
- double aggregate → Stores the aggregate marks of the student.

3: Which OOP principles does this program implement?

- **Encapsulation** → The Student class encapsulates the student data by using instance variables and accessing them through methods.

- **Abstraction** → The details of data insertion (insertStudent()) and retrieval (displayStudent()) are abstracted from the user.
- **Object-Oriented Approach** → Each Student object has its own copy of data, which follows the OOP paradigm.

4. Why do we use Scanner in the insertStudent() method?

- The Scanner class allows us to take **user input** from the console.
- It helps read different types of data like integers (nextInt()), strings (nextLine()), and doubles (nextDouble()).

5: What is the purpose of the Student class in this program?

The purpose of the Student class is to represent a **student entity** with properties like studentId, studentName, gender, department, age, and aggregate. It provides methods like insertStudent() for inserting details and displayStudent() for printing the stored information.